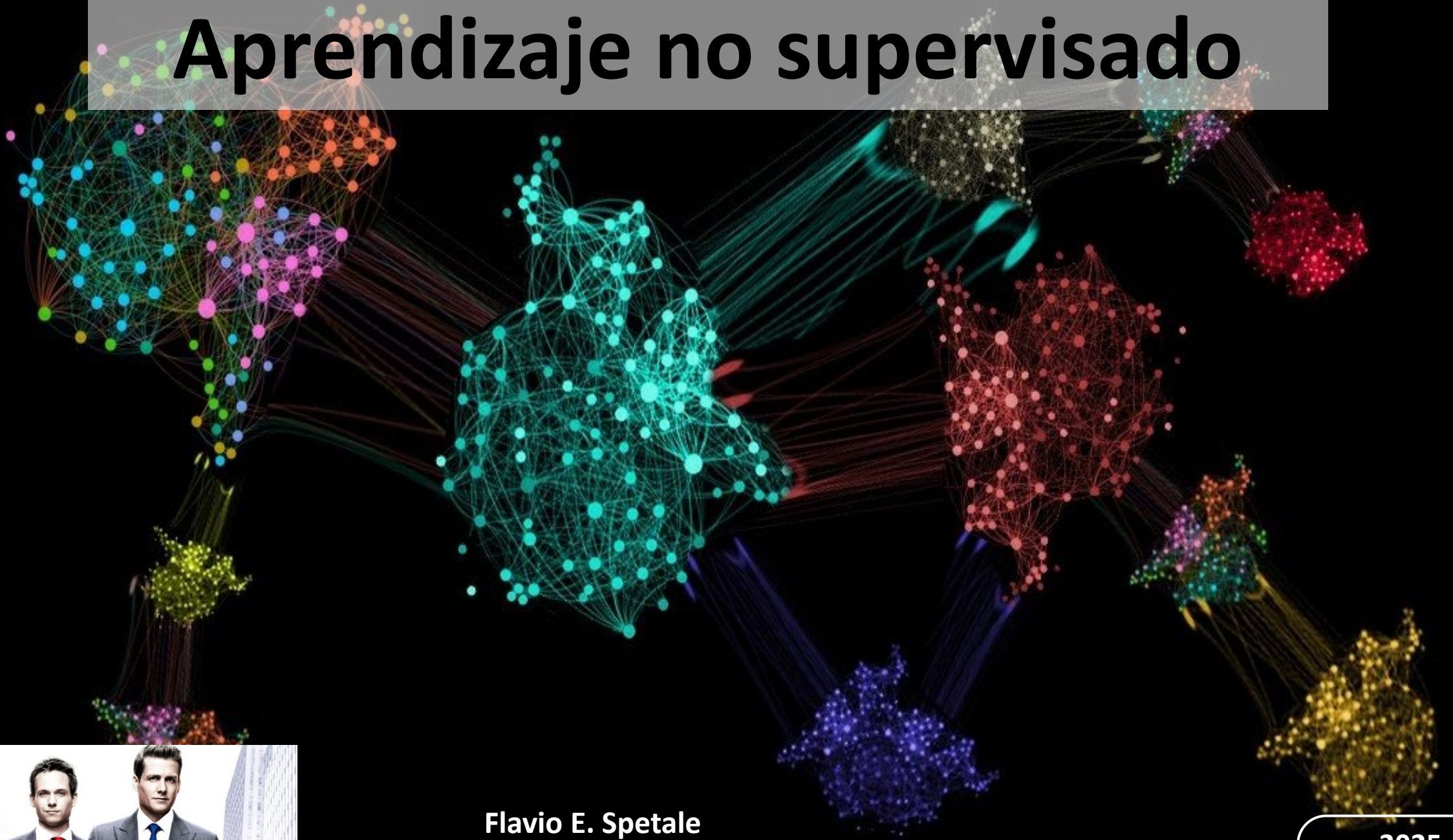


Aprendizaje no supervisado



Flavio E. Spetale

2025

Aprendizaje no supervisado

Es un tipo de aprendizaje automático donde un algoritmo busca *patrones y estructuras* en los datos detectados sin disponer de una variable objetivo ni de vigilancia humana.

A diferencia del Aprendizaje supervisado donde el algoritmo está entrenado para hacer una predicción numérica (regresión) o categórica (clasificación) basado en datos etiquetados, el proceso de aprendizaje no supervisado no usa datos etiquetados y busca estructuras/patrones en ellos por medio de similitudes que luego son agrupadas.

Las aplicaciones típicas del aprendizaje no supervisado son la segmentación de clientes en la investigación de marketing, la detección de anomalías en la Ciberseguridad o el reconocimiento de patrones en el tratamiento de imágenes y textos.

¿Cómo funciona el proceso del aprendizaje no supervisado?

En el aprendizaje automático no supervisado, el algoritmo debe encontrar de forma independiente correlaciones y patrones en los datos y utilizarlos para estructurar o agrupar los datos o para obtener nuevos conocimientos.

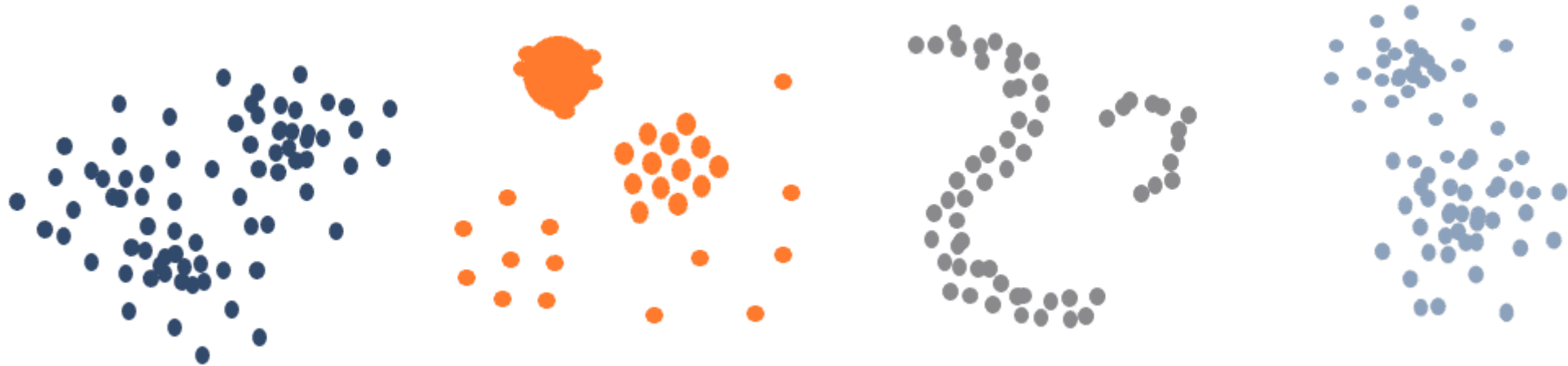
Los pasos del proceso son:

- El sistema interpreta los datos sin etiquetar, para identificar los patrones de organización que se encuentran ocultos.
- Encuentra los patrones ocultos, los cuales se fundamentan especialmente en las diferencias y las semejanzas encontradas.
- Finalmente, la máquina aplica los algoritmos correspondientes para dividir o segmentar en grupos de acuerdo con las semejanzas y diferencias encontradas entre ellos.

Tres tipos de métodos de aprendizaje no supervisado: Agrupación, Asociación y Reducción de la dimensionalidad.

Agrupación

El algoritmo agrupa los datos basándose en las similitudes de sus características en clusters (grupos). El objetivo de la agrupación es encontrar patrones en los datos y agruparlos para identificar la estructura intrínseca del conjunto de datos. Por ejemplo, el comportamiento de compra de los clientes de un supermercado puede analizarse para identificar grupos similares de compradores que adquieren productos parecidos. A este tipo de problemas se conoce como ***la maldición de la dimensionalidad***.



Algoritmos de agrupación

Algoritmo k-Means

El algoritmo k-means es uno de los algoritmos de agrupación más utilizados. Divide los datos en k grupos o clusters, donde k es el número de clusters dado. El algoritmo comienza con k centros seleccionados al azar y asigna el centro más cercano a cada punto de datos. A continuación, se vuelven a calcular los centros y se reasignan los puntos de datos. Este proceso se repite hasta que las asignaciones son estables.

Agrupación jerárquica

El Clustering Jerárquico es el proceso de agrupar gradualmente puntos de datos para crear una jerarquía de clusters. Existen dos tipos de Clustering Jerárquico: aglomerativo y divisivo. En el clustering aglomerativo, cada punto de datos comienza como un cluster separado. Estos clusters se combinan gradualmente para formar clusters más grandes. En cambio, en el clustering divisivo, el algoritmo parte de un cluster grande y lo divide en clusters más pequeños.

Algoritmos de agrupación

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) es un método de clustering basado en la densidad en el que el algoritmo intenta encontrar clusters de regiones densamente pobladas. Los puntos de datos en zonas densamente pobladas se consideran parte del mismo clúster, mientras que los puntos de datos en zonas poco pobladas se consideran valores atípicos o ruido.

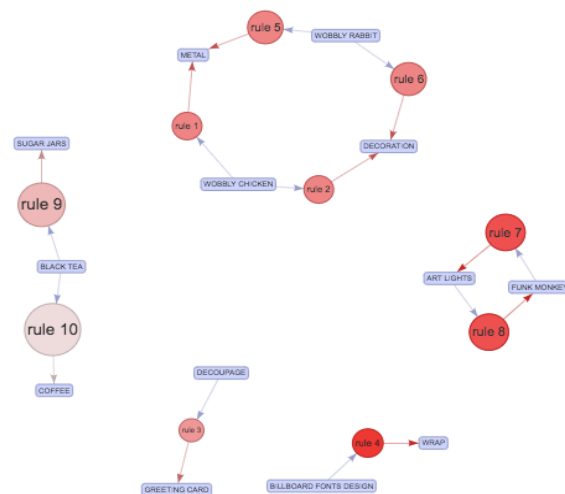
Desplazamiento medio

Mean Shift pretende identificar posibles centros de conglomerados en un conjunto de datos estimando la densidad de los datos y desplazando iterativamente los anchos de banda hacia la mayor densidad hasta alcanzar un punto de convergencia estable. A continuación, los puntos de datos de cada ancho de banda se asignan a un conglomerado. El desplazamiento de la media es especialmente útil para identificar conglomerados en conjuntos de datos con formas y tamaños diferentes y en los que la densidad de los puntos de datos no está distribuida homogéneamente.

Asociación

Se refiere al descubrimiento de patrones y relaciones comunes entre los distintos atributos de un conjunto de datos. La atención se centra en qué atributos o características del conjunto de datos aparecen juntos con frecuencia y cuáles no. El objetivo es identificar reglas o asociaciones que indiquen qué combinaciones de características o atributos se dan con más frecuencia.

Un ejemplo habitual de análisis de asociación es el análisis del comportamiento de compra en un supermercado. El objetivo es averiguar qué productos suelen comprarse juntos, por ejemplo para hacer recomendaciones a los clientes sobre futuras compras. Al identificar patrones y asociaciones, la empresa también puede optimizar la colocación de los productos en la tienda o dirigir la publicidad a productos específicos.



Algoritmos de asociación

Algoritmo Apriori

El algoritmo Apriori se utiliza principalmente en la investigación de mercados y el comercio electrónico. Identifica combinaciones de atributos que se dan con frecuencia y puede modificarse de varias formas para satisfacer diferentes requisitos de análisis de asociación.

El algoritmo funciona en dos pasos. En la primera, se analizan todos los elementos individuales del conjunto de datos y se determina la frecuencia de cada uno de ellos. En el segundo paso, se analizan las combinaciones de elementos (itemsets) para determinar la frecuencia de las combinaciones. El algoritmo utiliza un umbral de soporte para identificar los itemsets frecuentes que están por encima del umbral.

Algoritmos de asociación

Eclat

Eclat (Equivalence Class Clustering and bottom-up Lattice Traversal) es similar al algoritmo Apriori y utiliza un umbral de soporte para identificar los itemsets frecuentes. Sin embargo, a diferencia del algoritmo Apriori, Eclat no utiliza la generación de candidatos. En su lugar, utiliza una técnica de clases de equivalencia para determinar la frecuencia de los itemsets. Eclat se utiliza a menudo en la investigación de mercados y el comercio minorista para identificar correlaciones entre distintos productos o servicios.

Algoritmo FP-Growth

El algoritmo FP-Growth es otro algoritmo utilizado con frecuencia para el análisis de asociación. Sin embargo, a diferencia del algoritmo Apriori, no utiliza los principios de Apriori ni la generación de candidatos. En su lugar, el algoritmo crea un árbol (árbol FP) a partir de los itemsets del conjunto de datos. El árbol se utiliza para identificar todos los itemsets frecuentes del conjunto de datos. El algoritmo FP-Growth es rápido y eficaz porque sólo recorre el conjunto de datos una vez y utiliza el árbol FP para identificar los conjuntos frecuentes.

Reducción de la dimensionalidad

El objetivo de la reducción de la dimensionalidad es transformar el conjunto de datos a una dimensión inferior conservando la información importante. Esto puede ayudar a simplificar el conjunto de datos, reducir el tiempo de cálculo, reducir los requisitos de memoria y mejorar el rendimiento de los algoritmos de aprendizaje automático.

Por ejemplo, la reducción de la dimensionalidad en un conjunto de datos de clientes de un supermercado ayuda a reducir la variedad de características, como la edad, el sexo, los ingresos y los gastos. Esto ayuda a identificar las características más importantes y a explicar mejor la variación de los datos. Una vez extraídos los componentes principales, los clientes pueden visualizarse en un espacio bidimensional. Los clientes similares se sitúan cerca unos de otros, lo que puede indicar patrones y agrupaciones. Esto permite identificar segmentos de clientes y desarrollar estrategias de marketing u ofertas específicas.

Algoritmos de reducción de la dimensionalidad

Análisis de componentes principales (PCA)

El análisis de componentes principales pertenece a la extracción de características y suele utilizarse para analizar y visualizar datos. El objetivo del PCA es reducir la dimensionalidad de un conjunto de datos creando un nuevo conjunto de variables (denominadas componentes principales) que expliquen lo mejor posible la variación de los datos. Esto implica aplicar transformaciones lineales a los datos para encontrar una nueva base de sistema de coordenadas en la que la mayor varianza se encuentre en el primer componente principal, seguida de la segunda mayor varianza en el segundo componente principal, y así sucesivamente. El PCA puede ayudar a eliminar la información redundante de los datos, reducir las variables ruidosas y simplificar la estructura de los datos.

Análisis discriminante lineal (LDA)

El análisis discriminante lineal es un ejemplo de selección de características y se utiliza para clasificaciones. El objetivo del LDA es encontrar una combinación lineal de características que separe las clases minimizando la variación dentro de las clases. A diferencia del PCA, el LDA pretende proyectar los datos en un espacio de baja dimensión en el que las clases sean fácilmente distinguibles. LDA tiene en cuenta la pertenencia a una clase de los puntos de datos y optimiza la proyección para separar las clases lo mejor posible. Por tanto, el LDA puede utilizarse para seleccionar características o para crear otras nuevas que mejoren el rendimiento de la clasificación.

Algoritmos de reducción de la dimensionalidad

Incrustación estocástica de vecinos distribuida (t-SNE)

t-SNE puede utilizarse tanto para la extracción como para la selección de características. Se trata de un método no lineal para visualizar datos de alta dimensión en un espacio de baja dimensión. t-SNE es especialmente adecuado para captar relaciones y patrones complejos en los datos. Para ello, utiliza una distribución de probabilidad para calcular la similitud entre los puntos de datos en las dimensiones originales y en el espacio de baja dimensión. Las similitudes se modelan de tal forma que los puntos que están próximos entre sí en las dimensiones originales también lo están en la representación de baja dimensión. t-SNE puede ayudar a identificar conglomerados o agrupaciones en los datos y a visualizar estructuras de datos complejas.

Ventajas del aprendizaje no supervisado

- ***No se requieren datos etiquetados***: A diferencia del aprendizaje supervisado, no se necesitan datos de entrenamiento etiquetados. Esto puede ser muy útil cuando es difícil o caro obtener datos etiquetados.
- ***Detección de patrones ocultos***: Puede detectar patrones y estructuras ocultas en los datos que no son evidentes a primera vista. Esto puede ayudar a obtener nuevos conocimientos y perspectivas que de otro modo no se habrían descubierto.
- ***Detección de anomalías***: Puede detectar anomalías o valores atípicos en los datos que pueden indicar problemas o desviaciones. Esto puede ser útil en muchas aplicaciones, como la detección de fraudes, la seguridad o la vigilancia de la salud.
- ***Flexibilidad***: Es flexible y puede aplicarse de muchas formas distintas. Esto lo convierte en una herramienta versátil para el análisis de datos y el aprendizaje automático.
- ***Escalabilidad***: Puede aplicarse a grandes conjuntos de datos y suele ser escalable, lo que significa que puede utilizarse para problemas complejos y grandes conjuntos de datos.

Desventajas del aprendizaje no supervisado

- ***Dificultad en la evaluación:*** No hay variables objetivo, es más difícil evaluar el funcionamiento del algoritmo. No hay una forma clara de evaluar las predicciones del modelo, lo que hace que los resultados sean más difíciles de interpretar y validar.
- ***Interpretación errónea de los resultados:*** Es posible que el modelo detecte o interprete patrones incorrectos que carecen de significado. Si estos patrones se utilizan después como base para tomar decisiones, pueden ser inexactos o engañosos.
- ***Sobreajuste:*** Los modelos de aprendizaje no supervisado pueden Sobreajuste especialmente si el número de características de los datos es elevado. Así, el modelo puede detectar patrones que sólo están presentes en los datos de entrenamiento y no pueden generalizarse.
- ***Requiere conocimientos especializados:*** Los modelos de aprendizaje no supervisado suelen requerir cierto conocimiento experto para configurarse e interpretarse correctamente.

La maldición de la dimensionalidad

Como resultado, los conjuntos de datos de alta dimensión corren el riesgo de ser muy escasos: es probable que la mayoría de las instancias de entrenamiento estén lejos unas de otras. *Esto significa que una nueva instancia probablemente estará lejos de cualquier instancia de entrenamiento, lo que hace que las predicciones sean mucho menos confiables que en dimensiones más bajas, ya que se basarán en extrapolaciones mucho mayores.*

Cuanto más dimensiones tenga el conjunto de entrenamiento, mayor será el riesgo de sobreajuste.

En teoría, una solución a la maldición de la dimensionalidad podría ser aumentar el tamaño del conjunto de entrenamiento para alcanzar una densidad suficiente de instancias de entrenamiento. **Desafortunadamente, en la práctica, la cantidad de instancias de entrenamiento requeridas para alcanzar una determinada densidad crece exponencialmente con la cantidad de dimensiones.**

Ej: Con solo 100 características [0-1], en el problema MNIST (dígitos), necesitaría más instancias de entrenamiento que átomos en el universo observable para que las instancias de entrenamiento estén dentro de 0.1 entre sí en promedio, suponiendo que pudieran distribuirse uniformemente en todas las dimensiones.

K-means

El algoritmo está basado en la partición de un conjunto de n observaciones en k grupos en el que cada observación pertenece al grupo cuya distancia es menor.

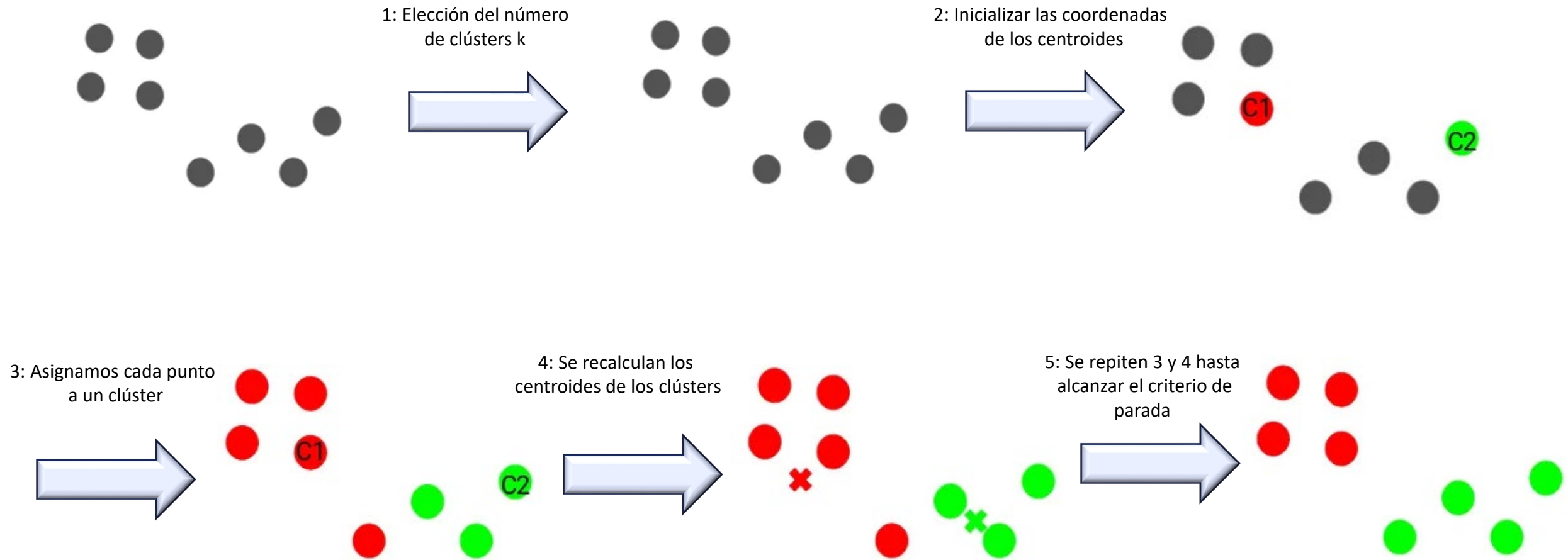
El agrupamiento se realiza minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster. Se suele usar la distancia cuadrática.

El algoritmo consta de tres pasos:

1. *Inicialización*: una vez escogido el número de grupos, k , se establecen k centroides en el espacio de los datos, por ejemplo, escogiéndolos aleatoriamente.
2. *Asignación objetos a los centroides*: cada objeto de los datos es asignado a su centroide más cercano.
3. *Actualización centroides*: se actualiza la posición del centroide de cada grupo tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo.

Se repiten los pasos 2 y 3 hasta que los centroides no se mueven, o se mueven por debajo de una distancia umbral en cada paso.

Algoritmo K-means



Algoritmo K-means

El algoritmo k-means resuelve un problema de optimización, siendo la función a optimizar (minimizar) la suma de las distancias cuadráticas de cada objeto al centroide de su cluster.

Los objetos se representan con vectores reales de d dimensiones $(x_1, x_2, \dots, x_n)$ y el algoritmo k-means construye k grupos donde se minimiza la suma de distancias de los objetos, dentro de cada grupo $S=\{S_1, S_2, \dots, S_k\}$, a su centroide. El problema se puede formular de la siguiente forma:

$$\min_S E(\mu_i) = \min_S \sum_{i=1}^k \sum_{\mathbf{x}_j \in S_i} \|\mathbf{x}_j - \mu_i\|^2 \quad (1)$$

donde S es el conjunto de datos cuyos elementos son los objetos x_j representados por vectores, donde cada uno de sus elementos representa una característica o atributo. Tendremos k grupos con su correspondiente centroide μ_i .

En cada actualización de los centroides, desde el punto de vista matemático, imponemos la condición necesaria de extremo a la función $E(\mu_i)$ que, para la función cuadrática (1) es:

$$\frac{\partial E}{\partial \mu_i} = 0 \implies \mu_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{\mathbf{x}_j \in S_i^{(t)}} \mathbf{x}_j$$

y se toma el promedio de los elementos de cada grupo como nuevo centroide.

Criterio de parada para K-means

Existen tres criterios:

1. *Los centroides dejan de cambiar*, i.e., después de múltiples iteraciones, los centroides de cada clúster no cambian. Por lo que se asume que el algoritmo ha convergido.
2. *Los puntos dejan de cambiar de clúster*. No se fijan en los centroides, si no en los puntos que pertenecen a cada clúster. Cuando se observa que no hay un intercambio de clústers se asume que el modelo está entrenado.
3. *Límite de iteraciones*. Se fija un número máximo de iteraciones que queremos que nuestro algoritmo ejecute antes de pararlo. Cuando llega a ese número, se asume que el modelo no va a mejorar drásticamente y se para el entrenamiento.

En la práctica, no es necesario que el método converja, y generalmente es suficiente cuando se tiene pequeños cambios en la pertenencia de las instancias en los distintos grupos o particiones, es decir, cuando estamos próximos a una situación de convergencia.

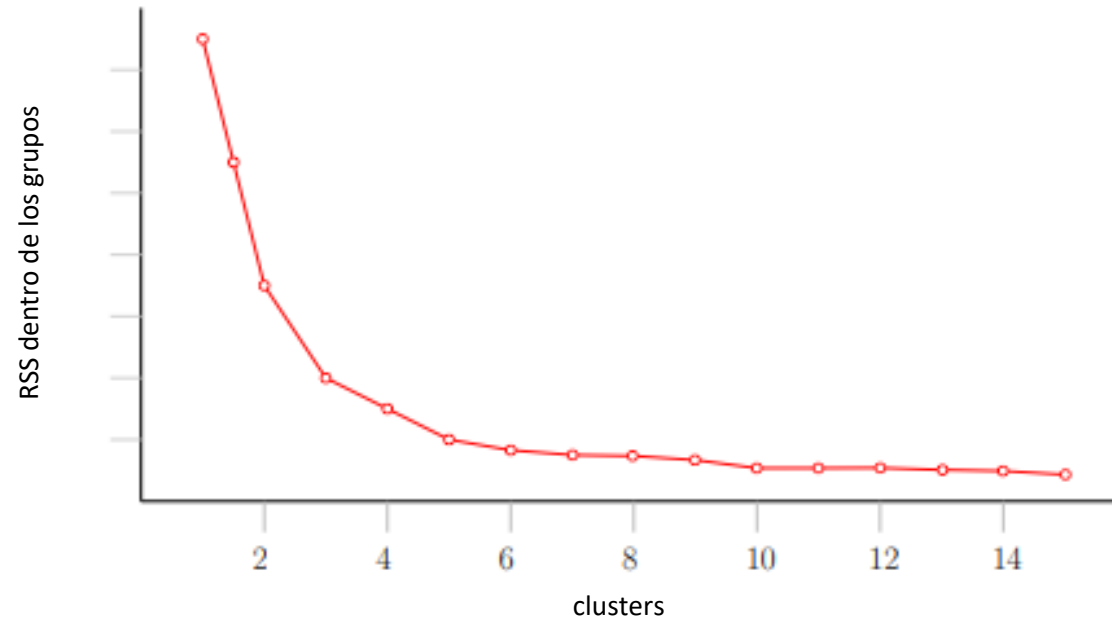
Criterios para seleccionar un valor k

1. Minimizar la suma de residuos cuadrados (RSS), es decir, la suma de las distancias de cualquier vector o instancia a su centroide más cercano. Este criterio busca la creación de segmentos lo más compactos posibles.
2. Maximizar la suma de distancias entre segmentos, por ejemplo entre sus centros. En este caso se estaría priorizando tener segmentos los más alejados posible entre sí, i.e., tener segmentos lo más diferenciados posible.

Como sucede en muchos problemas de maximización o minimización, es fácil intuir que el algoritmo no siempre alcanzará un óptimo global, puede darse el caso de que antes encuentre un óptimo local. Además, también es posible que el algoritmo no sea capaz de encontrar ningún óptimo, bien porque simplemente el juego de datos no presente ninguna estructura de segmentos, bien porque no se haya escogido correctamente el número k de segmentos a construir.

Los criterios anteriores (minimización de distancias intragrupo o maximización de distancias intergrupo) pueden usarse para establecer un valor adecuado para el parámetro k . Valores k para los que ya no se consiguen mejoras significativas en la homogeneidad interna de los segmentos o la heterogeneidad entre segmentos distintos, deberían descartarse.

Criterios para seleccionar un valor k



Se observa cómo a partir de cinco segmentos, la mejora que se produce en la distancia interna de los segmentos ya es insignificante. Este hecho debería ser indicativo de que cinco segmentos es un valor adecuado para k.

Criterios para seleccionar un valor k

Otra aproximación para solucionar este problema consiste en permitir que el número k se modifique durante la ejecución del algoritmo. En este caso, se inicia el algoritmo con un valor proporcionado de k , pero este se puede modificar (incrementar o disminuir) según ciertos criterios.

1. Si existen dos particiones con los centros muy juntos, quizás es mejor unir ambas particiones y reducir el número de particiones.
2. Si existe una partición con un nivel de diversidad muy elevado, se puede dividir esta en dos nuevas particiones, incrementando por tanto el valor de k .

Ejemplo en python

Conjunto de datos: *Clasificación de variedades de trigo*

Información del conjunto de datos:

El conjunto de datos comprendía granos de trigo pertenecientes a tres variedades diferentes de trigo: Kama, Rosa y Canadian, 70 elementos cada una.

Todos estos parámetros fueron de valor real continuo.

Información de atributos:

Para construir los datos, se midieron siete parámetros geométricos de los granos de trigo:

área A, perímetro P, compacidad $C = 4\pi A/P^2$, longitud del grano, ancho del grano, coeficiente de asimetría y longitud del surco del grano.

Ejemplo en python

```
OMP_NUM_THREADS=1
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from sklearn.cluster import Kmeans
from gap_statistic import OptimalK
```

```
target_names = {
    1:'Kama',
    2:'Rosa',
    3:'Canadian'
}
```

```
dataWheat = pd.read_csv("wheat.csv")
dataWheat['category'] = dataWheat['category'].map(target_names)
```

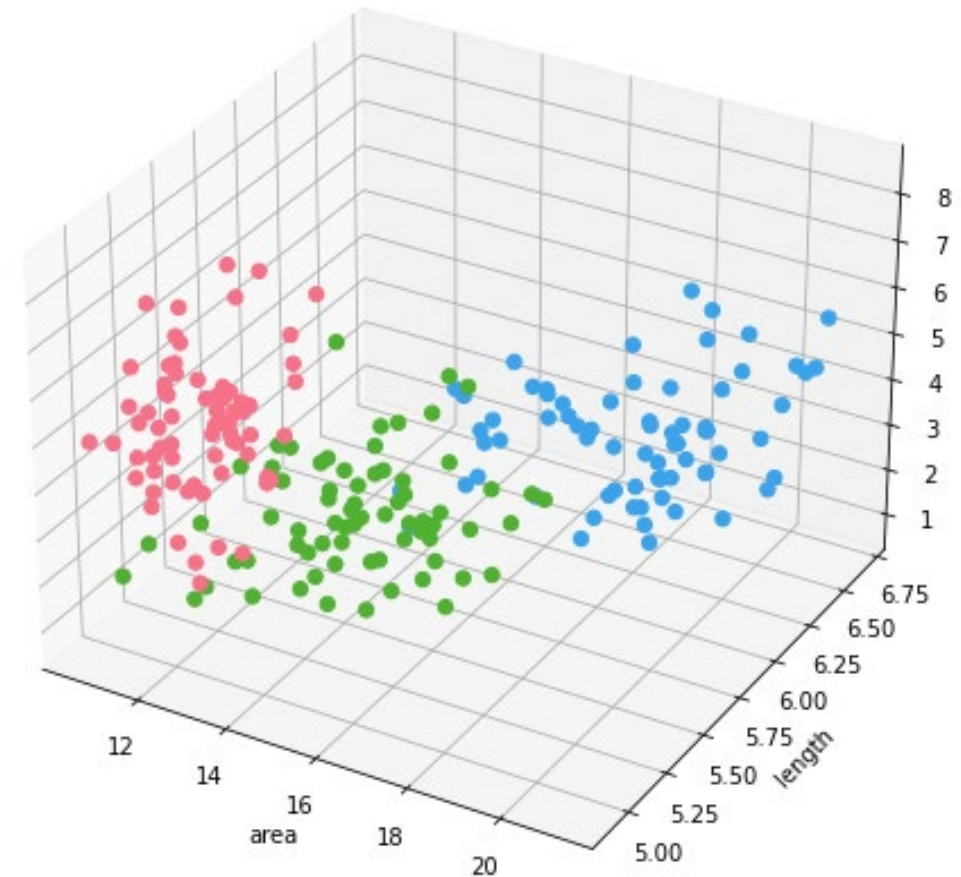
```
xWheat = np.array(dataWheat[["area", "length", "asymmetry coefficient"]])
yWheat = np.array(dataWheat['category'])
```

Ejemplo en python

```
fig = plt.figure(figsize=(10, 6))
ax = Axes3D(fig, auto_add_to_figure=False)
fig.add_axes(ax)

labels = np.unique(dataWheat['category'])
palette = sns.color_palette("husl", len(labels))

for label, color in zip(labels, palette):
    df1 = dataWheat[dataWheat['category'] == label]
    ax.scatter(df1['area'], df1['length'], df1['asymmetry coefficient'],
              s=40, marker='o', color=color, alpha=1, label=label)
ax.set_xlabel('area')
ax.set_ylabel('length')
ax.set_zlabel('asymmetry coefficient')
```

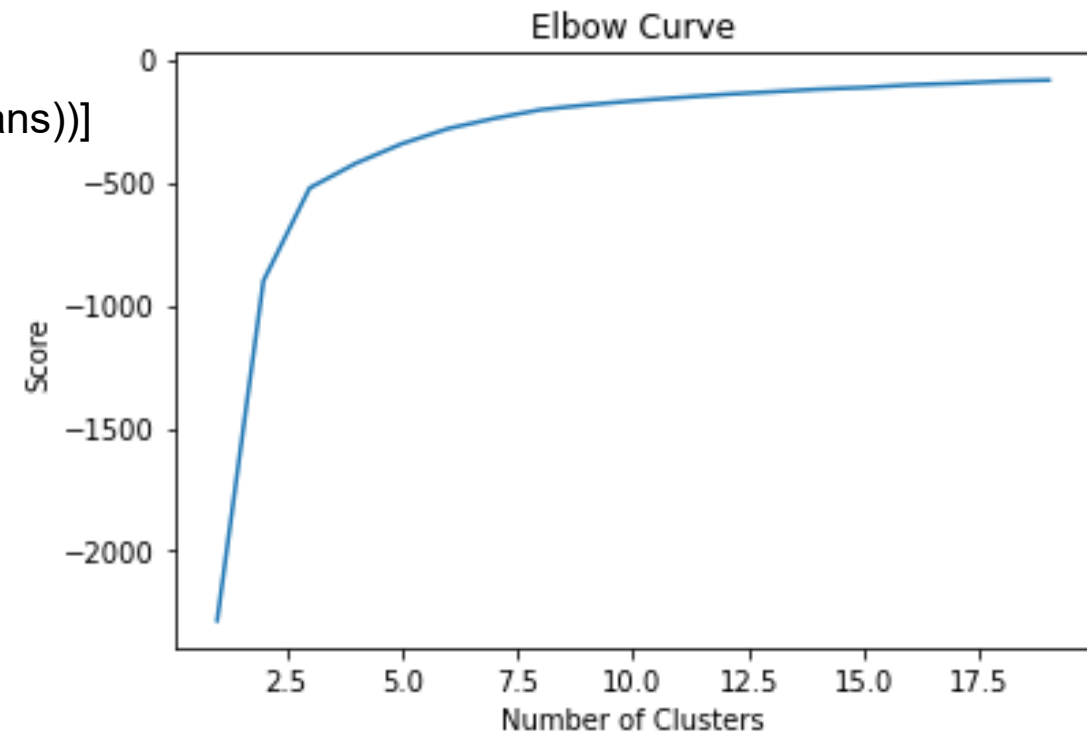


Ejemplo en python

```
""" K-means """
```

```
Nc = range(1, 20)
kmeans = [KMeans(n_clusters=i) for i in Nc]
score = [kmeans[i].fit(xWheat).score(xWheat) for i in range(len(kmeans))]
```

```
plt.plot(Nc,score)
plt.xlabel('Number of Clusters')
plt.ylabel('Score')
plt.title('Elbow Curve')
plt.show()
```



```
kmeans = KMeans(n_clusters=5).fit(xWheat)
centroids = kmeans.cluster_centers_
```

```
print(centroids)
```

```
[[14.78074074  5.58187037  2.42596667]
 [19.19181818  6.27129545  3.25363636]
 [12.09045455  5.21740909  3.34375   ]
 [16.92461538  5.95884615  4.51838462]
 [11.9847619   5.24138095  5.6732619 ]]
```

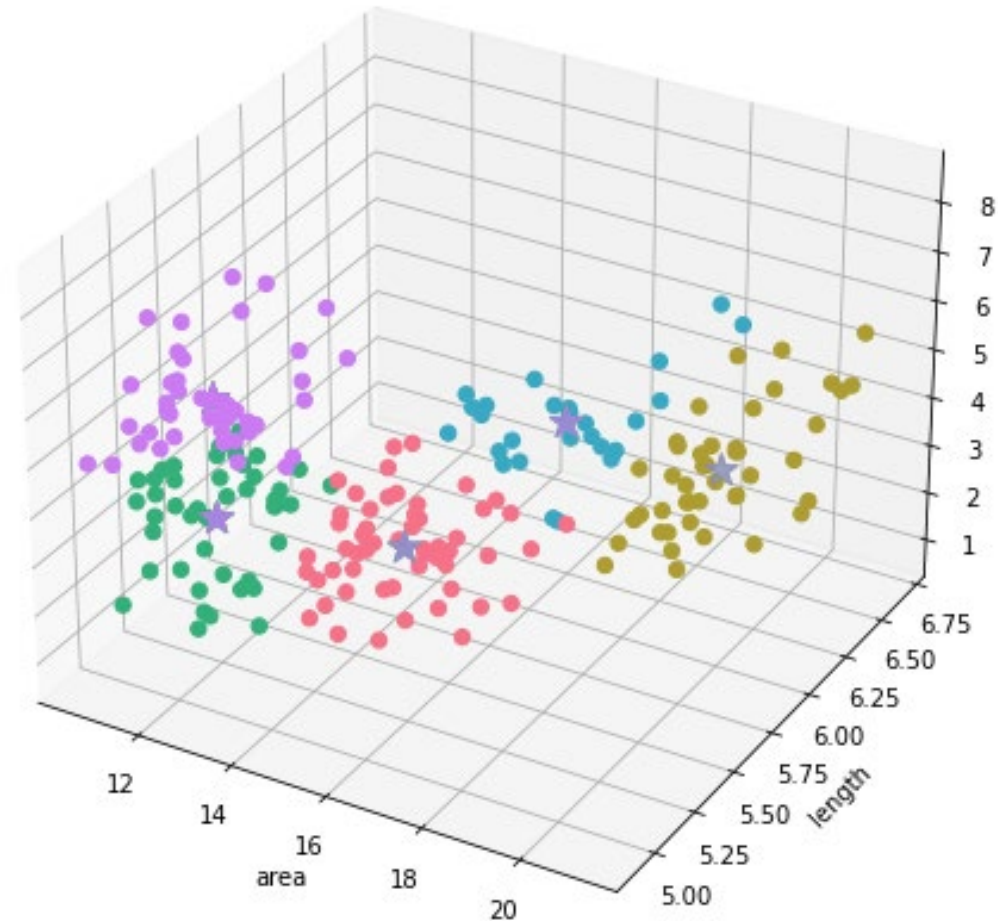
Ejemplo en python

```
centerClusters = kmeans.cluster_centers_
```

```
fig = plt.figure(figsize=(10, 6))  
ax = Axes3D(fig, auto_add_to_figure=False)  
fig.add_axes(ax)
```

```
dataWheat['newClass'] = kmeans.predict(xWheat)  
labels = np.unique(dataWheat['newClass'])  
palette = sns.color_palette("husl", len(labels))
```

```
for label, color in zip(labels, palette):  
    df1 = dataWheat[dataWheat['newClass'] == label]  
    ax.scatter(df1['area'], df1['length'], df1['asymmetry coefficient'],  
              s=40, marker='o', color=color, alpha=1, label=label)  
    ax.scatter(centerClusters[:, 0], centerClusters[:, 1], centerClusters[:, 2],  
              marker='*', color=color, s=200)  
ax.set_xlabel('area')  
ax.set_ylabel('length')  
ax.set_zlabel('asymmetry coefficient')
```



Clustering jerárquico

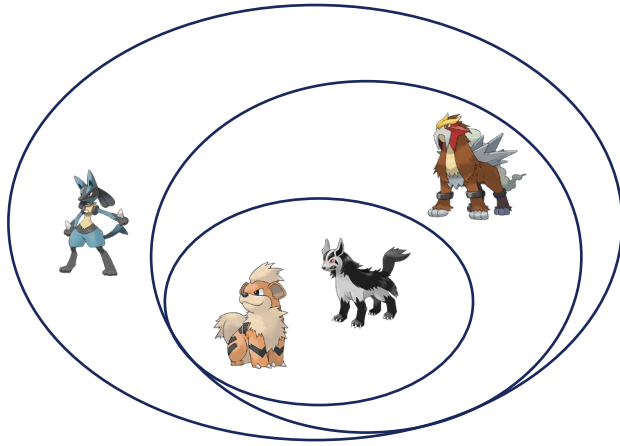
Es un método que busca construir una jerarquía de particiones o clústeres en el conjunto de datos.

Hay dos tipos de algoritmos de agrupamiento jerárquico que, aunque pueden proporcionar resultados similares, tienen enfoques inversos:

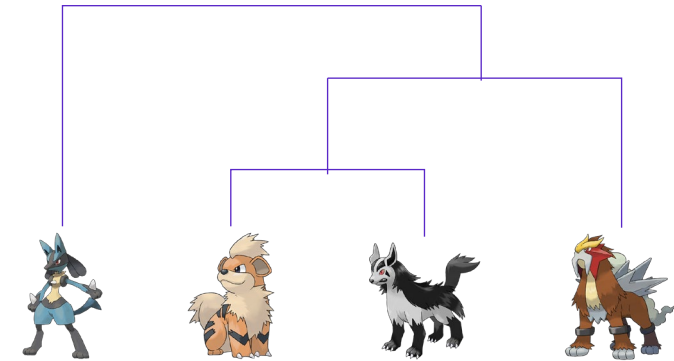
1. El algoritmo aglomerativo (Agglomerative Hierarchical Clustering, AHC) parte de una fragmentación completa de los datos, es decir, cada dato tiene su propio grupo, y fusiona los grupos progresivamente hasta que todos los datos pertenecen a un único grupo. Aplica una estrategia bottom-up. En este caso hablaremos de clustering o agrupamiento.
2. El algoritmo divisivo (Divisive Hierarchical Clustering, DHC) parte de un único grupo que contiene todos los datos y lo va dividiendo de forma recursiva hasta obtener un grupo para cada dato. Es decir, aplica una estrategia top-down. En este caso hablaremos de segmentación.

El principal inconveniente de este tipo de algoritmos es que no son capaces de determinar, por sí mismos, el número idóneo de grupos a generar, sino que es necesario fijarlos de antemano, o bien definir algún criterio de parada en el proceso de construcción jerárquica.

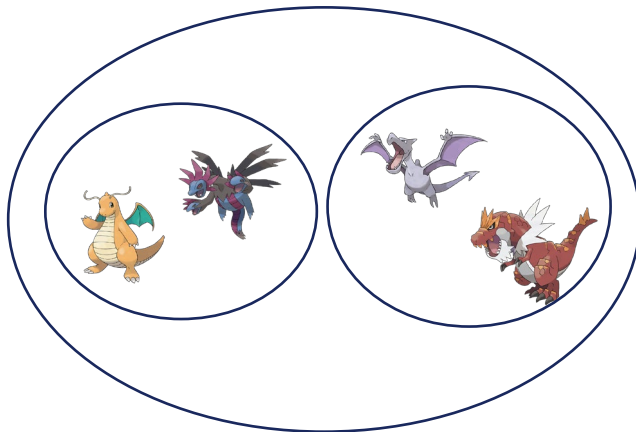
Clustering jerárquico vs Diagramas de Venn



Diagramas de Venn



Clustering Jerárquico



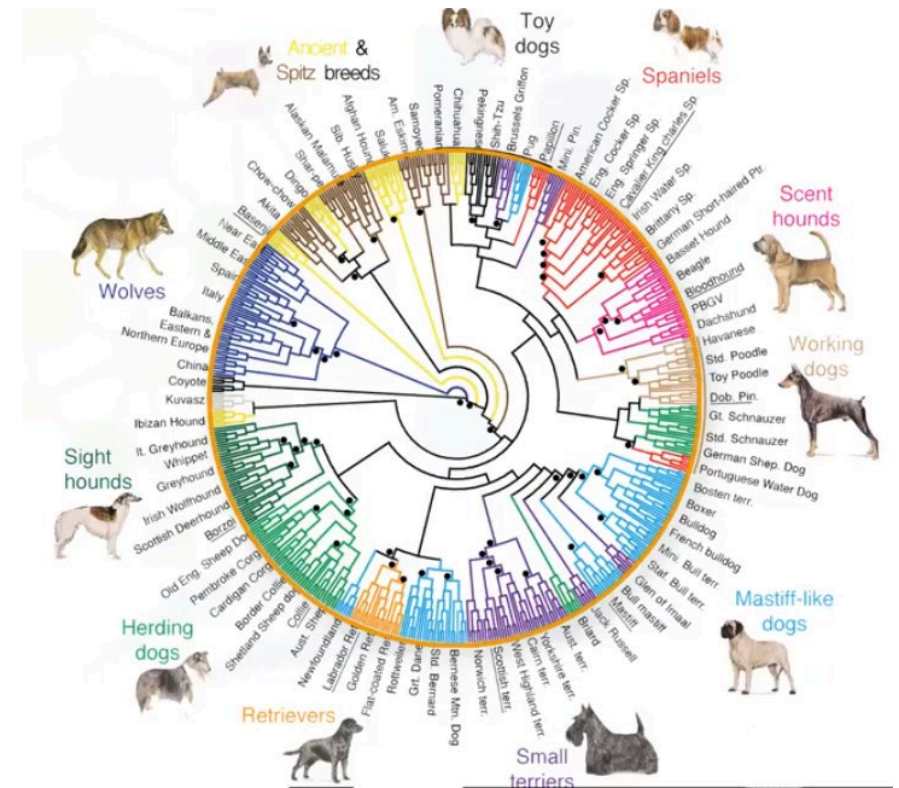
Dendrogramas

El dendrograma es un diagrama que muestra las agrupaciones (divisiones) sucesivas que genera un algoritmo jerárquico. Sin duda, es una forma muy intuitiva de representar el proceso de construcción de los grupos.

Sin embargo, un punto débil es no ofrecer información sobre la distancia entre los distintos objetos, aunque se puede utilizar el tamaño de las flechas o trabajar con tonalidades de un mismo color para añadir este tipo de información.

El principal interés de los algoritmos jerárquicos es que generan *diferentes niveles de agrupamiento*, lo que proporciona una información igual o más útil, en algunos casos, que la obtención del agrupamiento definitivo.

Es importante destacar que los algoritmos jerárquicos son voraces (greedy), lo que significa que *buscan la mejor decisión local en cada momento, sin asegurar una solución global óptima*.

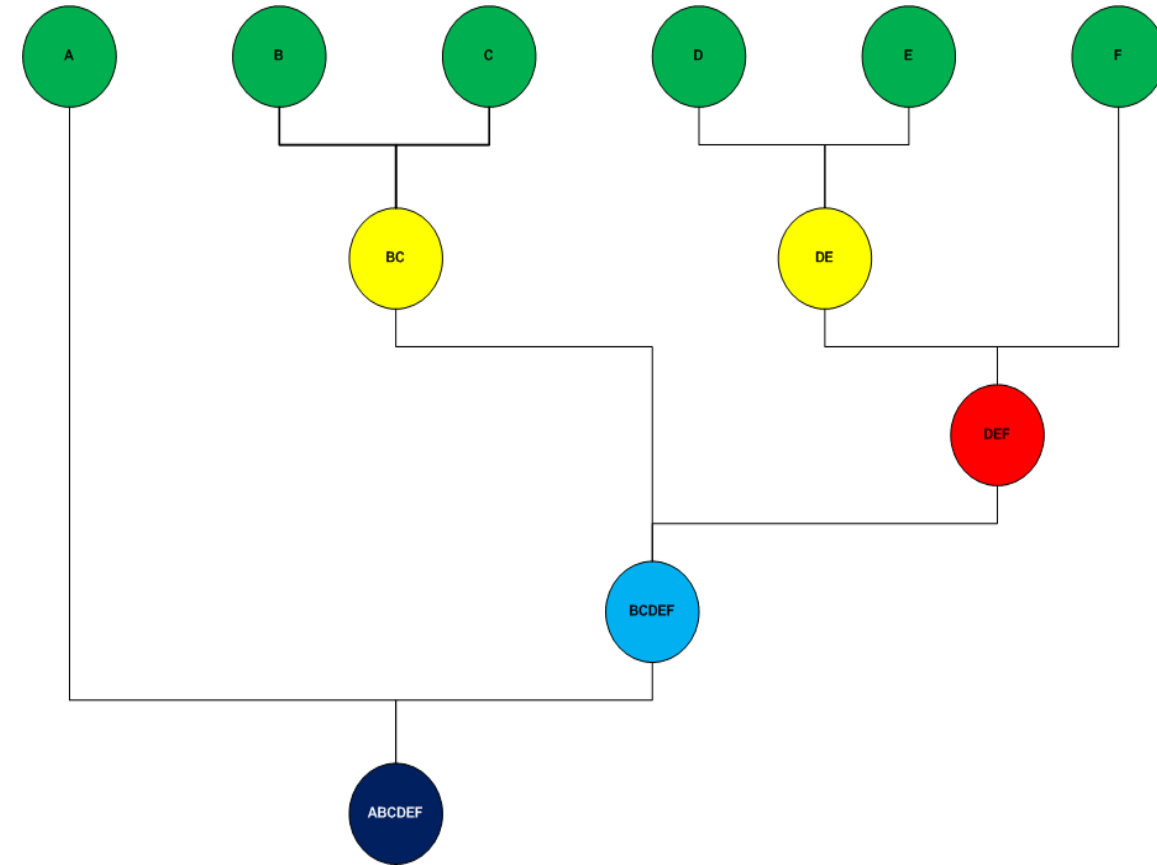


Algoritmos aglomerativos

Es una aproximación de abajo hacia arriba (bottom-up) donde se dividen los clusters en subclusters y así sucesivamente. Iniciando asignando cada muestra simple a un cluster y en cada iteración sucesiva va aglomerando (mezclando) el par de clusters más cercanos satisfaciendo algún criterio de similitud, hasta que todos los elementos pertenecen a un solo cluster. Los clusters generados en los primeros pasos son anidados con los clusters generados en los siguientes pasos.

Algoritmo:

1. Asignar cada observación a un cluster individual
2. Calcular la matriz de distancias entre todos los puntos/clusters
3. Seleccionar los 2 clusters más cercanos según un criterio de distancia específico
4. Fusionar los clusters seleccionados
5. Actualizar la matriz de distancias
6. Repetir los pasos 3-5 hasta formar un único cluster grande

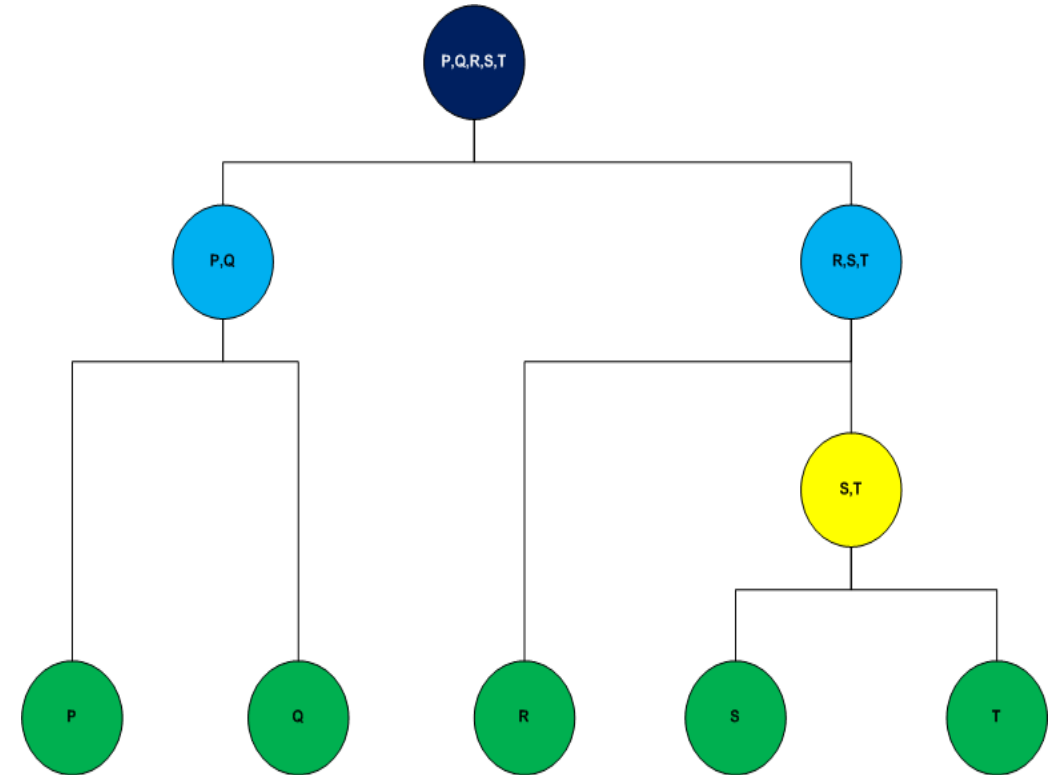


Algoritmos divisivos

Este tipo de clustering se lleva a cabo con un enfoque de arriba hacia abajo (top-down), Se inicia con todos los elementos asignado a un solo cluster y sigue el algoritmo hasta que cada elemento es un cluster individual.

A diferencia del enfoque de abajo hacia arriba donde las decisiones para generar los clusters se basan en lo patrones locales sin tomar en cuenta la distribución global, el enfoque de arriba hacia abajo se beneficia de la información completa sobre la distribución global al ir haciendo las particiones.

Aunque el agrupamiento aglomerativo representa la forma más natural de construir un dendograma, su costo computacional es alto. El agrupamiento divisivo suele ser más eficiente a nivel computacional, principalmente porque no se suele llegar al nivel mínimo del dendograma (una clase por dato), dado que no aporta información útil. Se define un criterio de parada que determinará la profundidad o nivel máximo del árbol.



Criterios para determinar número de clusters

1. La profundidad máxima del árbol resultante.
2. El número máximo de instancias que deberán de tener las particiones. Las particiones con cardinalidad inferior ya no serán procesadas otra vez.
3. Un valor máximo para una medida de similaridad. Las particiones que no superen este umbral, no serán divididas otra vez, ya que presentan un nivel de cohesión suficiente

NO existe un óptimo absoluto único

1. No existe un criterio objetivo ni ampliamente válido para la elección de un número óptimo de Clusters
2. La optimalidad depende del contexto y objetivo del análisis
3. Diferentes criterios pueden sugerir valores diferentes de k

Métodos de división de clústeres

Supongamos que tenemos dos clusters A y B, formados por puntos $a_i \in A$ y $b_j \in B$

Single – linkage (SLCA): la distancia entre dos clústeres es la distancia mínima que existe entre dos elementos que pertenecen a diferentes clústeres.

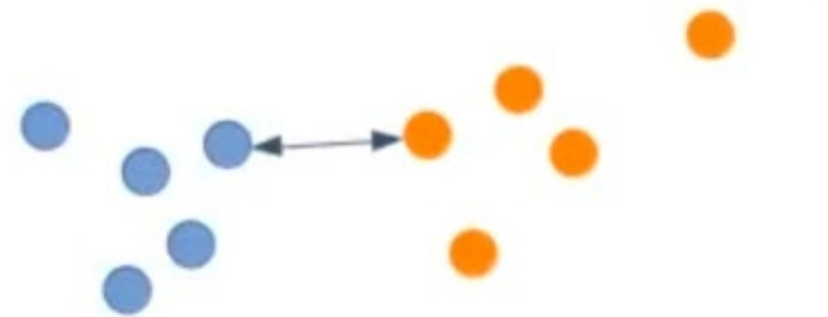
$$D(A, B) = \min_{a_i \in A, b_j \in B} d(a_i, b_j)$$

SLCA suele ser poco estable. No es muy buena opción, porque a lo mejor tienes ruido entre clústeres y/o las fronteras son muy difusas, de modo que este modelo puede hacer que, por puro azar, un punto esté más cerca o más lejos de otro.

Complete – linkage (CLCA): la distancia entre dos clústeres es la distancia máxima que existe entre dos elementos que pertenecen a diferentes clústeres.

$$D(A, B) = \max_{a_i \in A, b_j \in B} d(a_i, b_j)$$

Este método suele ser más estable, más robusto. No obstante, presenta un problema y es que suele ser bastante pesimista.



Métodos de división de clústeres

Average – linkage (unweighted pair group method with arithmetic mean, UPGMA): la distancia entre dos clústeres es la distancia promedio que existe desde cualquier elemento en el primer clúster a cualquier elemento en el segundo clúster.

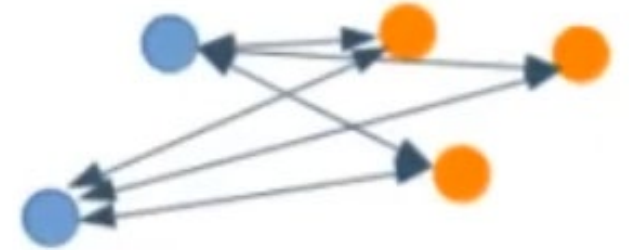
$$D(A, B) = \frac{1}{|A||B|} \sum_{a_i \in A} \sum_{b_j \in B} d(a_i, b_j)$$

Hace un balance: ni tan alargado como single, ni tan compacto como complete.

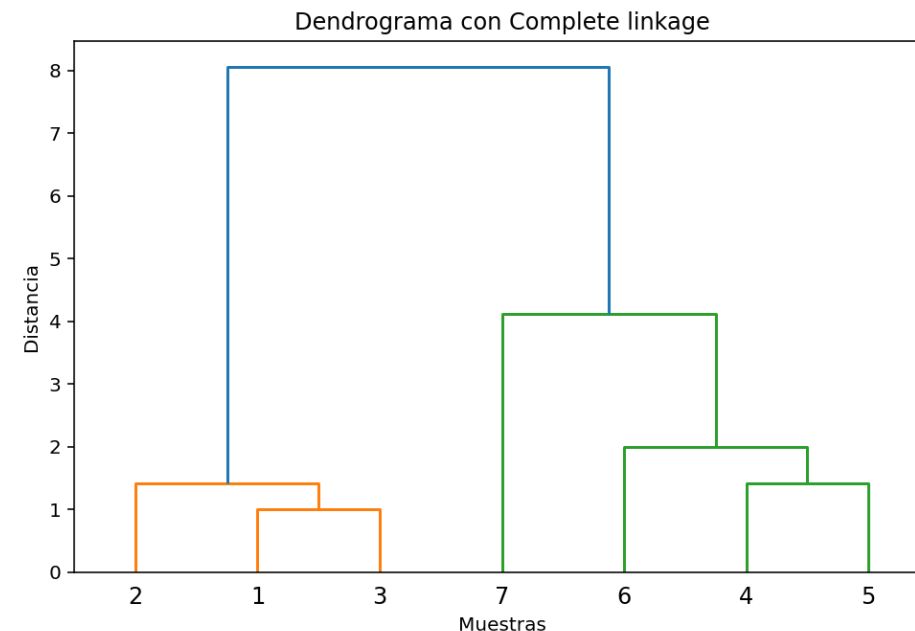
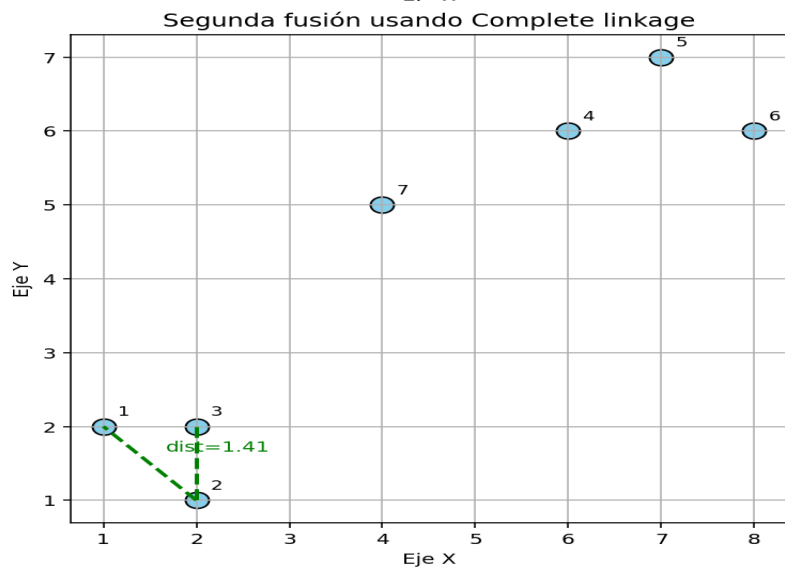
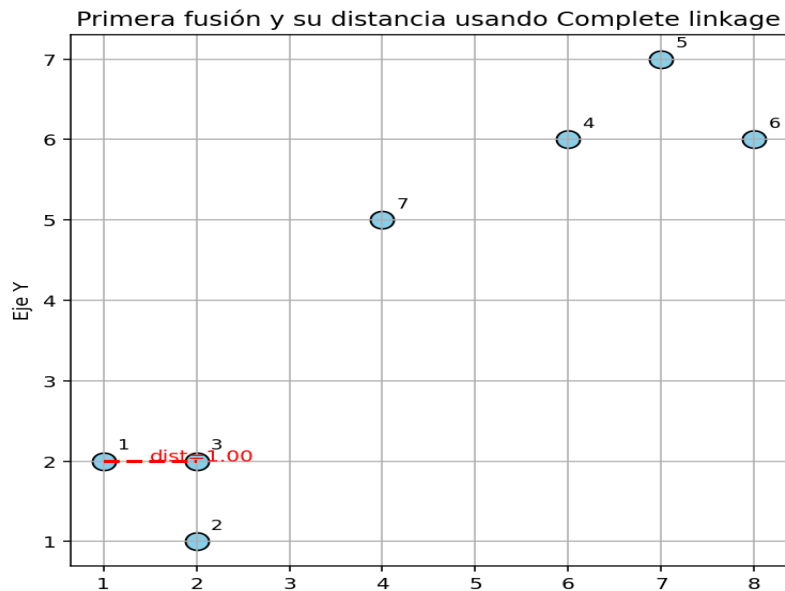
Centroid – linkage: (Unweighted pair group method with centroid, UPGMC): la distancia entre dos clústeres es la distancia que existe entre los centroides de cada clúster.

$$C_A = \frac{1}{|A|} \sum_{a_i \in A} a_i \quad \vee \quad C_B = \frac{1}{|B|} \sum_{b_j \in B} b_j \quad \therefore D(A, B) = d(C_A, C_B)$$

Puede producir el fenómeno inverso (clusters previamente unidos pueden volver a separarse).



Ejemplo 2D: Clustering jerárquico



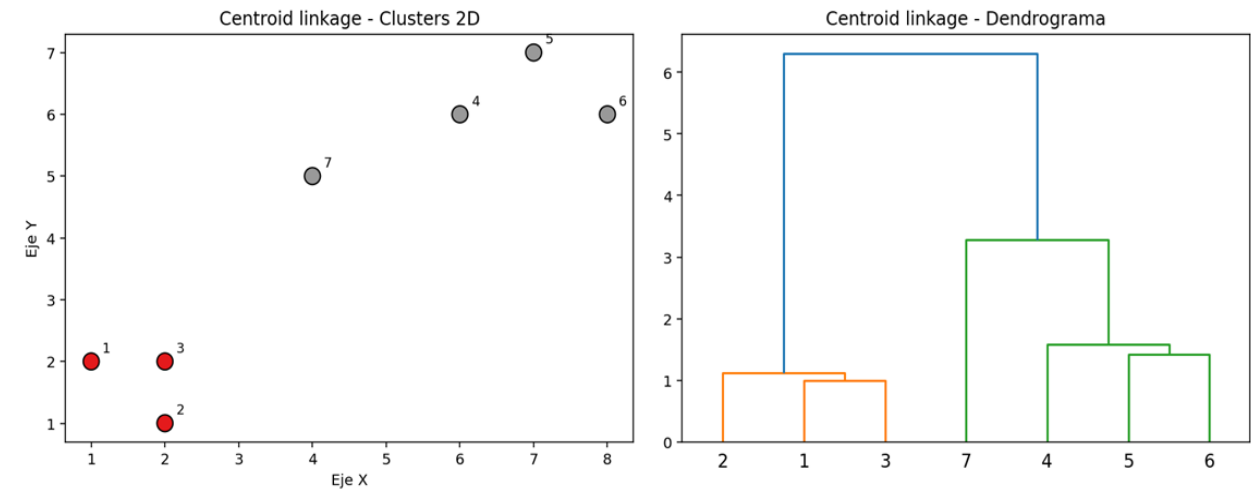
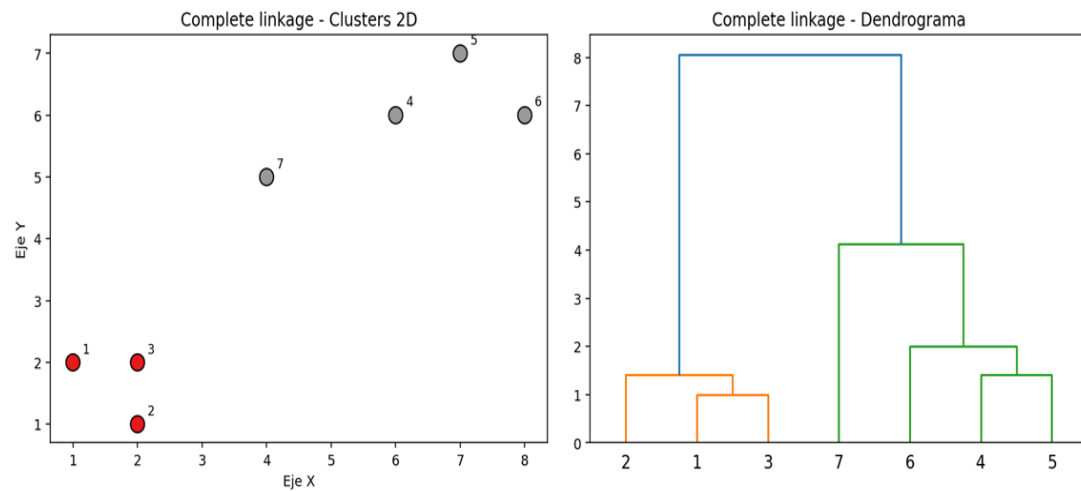
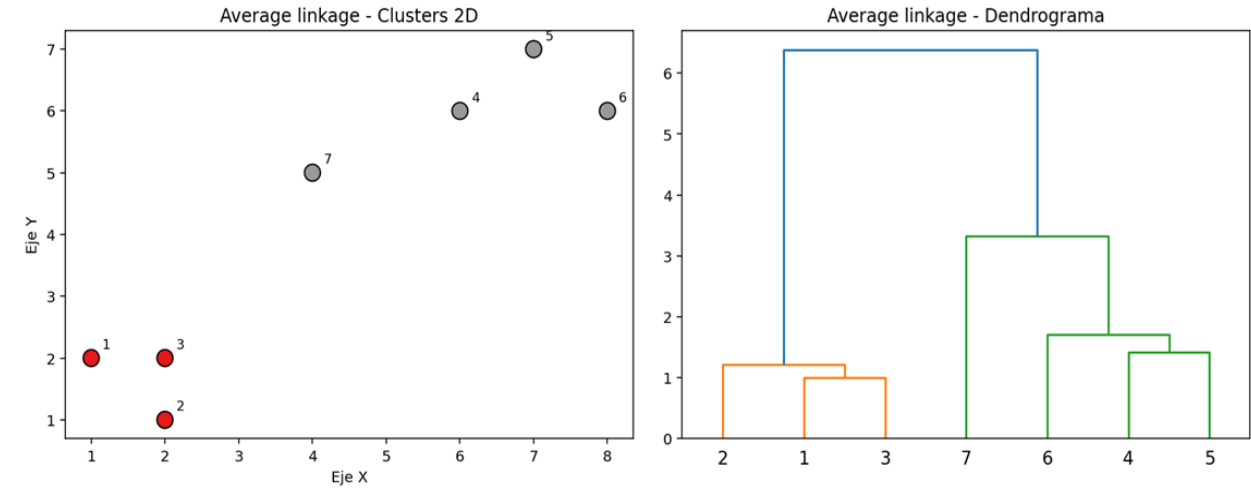
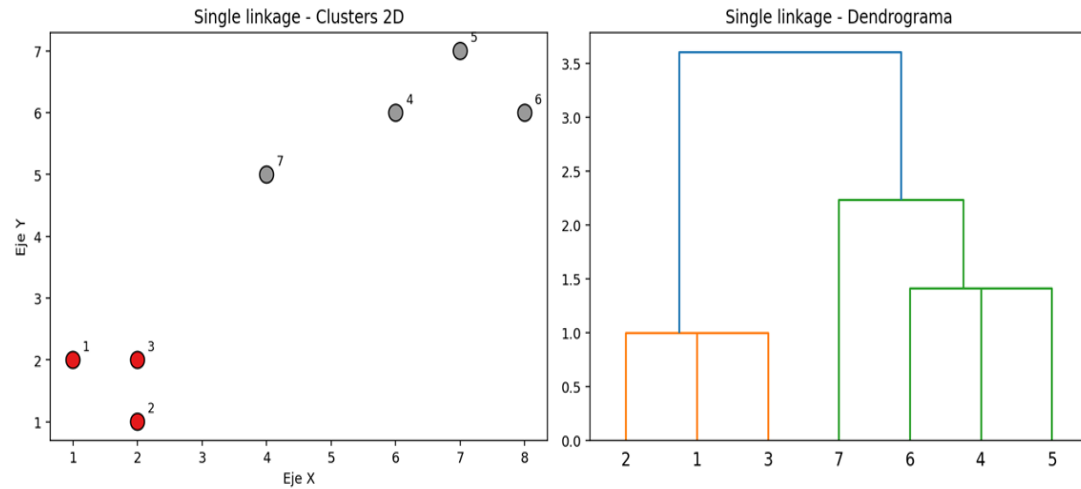
Eje X: representa las muestras o clusters iniciales.

Eje Y: representa la **distancia o disimilaridad** en la que se fusionan los clusters.

Cuanto **más baja** es la unión → los cluster están muy próximos (más similares).

Cuanto **más alta** es la unión → se unieron dos grupos que están más lejanos (menos similares).

Ejemplo 2D: Clustering jerárquico



Tipos de Validación

La validación externa y la validación interna son las dos categorías más importantes para la validación de clustering. La principal diferencia es si se usa o no información externa para la validación, i.e., información que no es producto de la técnica de agrupación utilizada.

A diferencia de técnicas de validación externas, las de validación interna miden el clustering únicamente basadas en información de los datos. Evalúan que tan buena es la estructura del clustering sin necesidad de información ajena al propio algoritmo y su resultado.

Como la validación externa mide la calidad del agrupamiento conociendo información externa de antemano, es principalmente usada para escoger un algoritmo de clustering óptimo sobre un conjunto de datos específico.

Las métricas de validación interna pueden usarse para escoger el mejor algoritmo de clustering, así como el número de clúster óptimo sin ningún tipo de información adicional.

En la práctica, la información externa por lo general no se encuentra disponible en muchos escenarios de aplicación.

Métricas de Validación Interna

Como el objetivo del clustering es agrupar objetos similares en el mismo clúster y objetos diferentes ubicarlos en diferentes clúster, las métricas de validación interna están basadas usualmente en los dos siguientes criterios:

- *Cohesión*: El miembro de cada clúster debe ser lo más cercano posible a los otros miembros del mismo clúster.
- *Separación*: Los clúster deben estar ampliamente separados entre ellos. Existen varios enfoques para medir esta distancia entre clúster: distancia entre el miembro más cercano, distancia entre los miembros más distantes o la distancia entre los centroides.

Índices internos

La mayoría de los índices internos se basan en la propiedad de que un buen algoritmo de clustering genera clusters con alta concentración interna y buena separación externa.

La suma de cuadrados intraclusters (Sum of Squares Within - SSW) es comúnmente utilizada para medir la compactación de los clusters, evaluando la variabilidad o dispersión de los puntos dentro de cada cluster. Por otro lado, la suma de cuadrados interclusters (Sum of Squares Between - SSB) mide la separación entre los diferentes clusters, cuantificando la distancia entre los centroides de los clusters.

$$SSW(k) = \sum_{i=1}^k \sum_{x \in C_i} ||x - m_i||^2$$
$$SSB(k) = \sum_{i=1}^k |C_i| ||m_i - M||^2$$

donde k es la cantidad de clusters, m_i corresponde al centro del cluster C_i , $|C_i|$ es el tamaño del cluster C_i y M es el centroide de los centros de los k clusters.

Índices internos

Gap statistic se usa para estimar el numero de clusters comparando la curva del logaritmo de la dispersión intracluster obtenida a partir de los datos, con el valor correspondiente esperado bajo una distribución uniforme generada sobre el rectángulo que contiene a los datos.

Intuitivamente utiliza el fenómeno del codo (elbow) en la curva de dispersión de la suma intracluster. Gap statistic esta diseñado para ser aplicado a cualquier técnica de clustering y medida de distancia.

$$Gap(k) = \frac{1}{B} \left(\sum_{b=1}^B \log(W_{kb}^*) \right) - \log(W_k)$$

donde $\frac{1}{B} \sum \log(W_{kb}^*)$ indica el valor esperado de $\log(W_k)$ calculado a partir de una distribución de referencia, B son la cantidad de repeticiones donde se considero la distribución de referencia

Índices internos

El coeficiente o puntuación de silueta, Silhouette, es una métrica que se utiliza para calcular la bondad de una técnica de agrupación. Su valor oscila entre -1 y 1.

1: Significa que los grupos están bien separados unos de otros y claramente distinguidos.

0: Significa que los clusters son indiferentes, o podemos decir que la distancia entre clusters no es significativa.

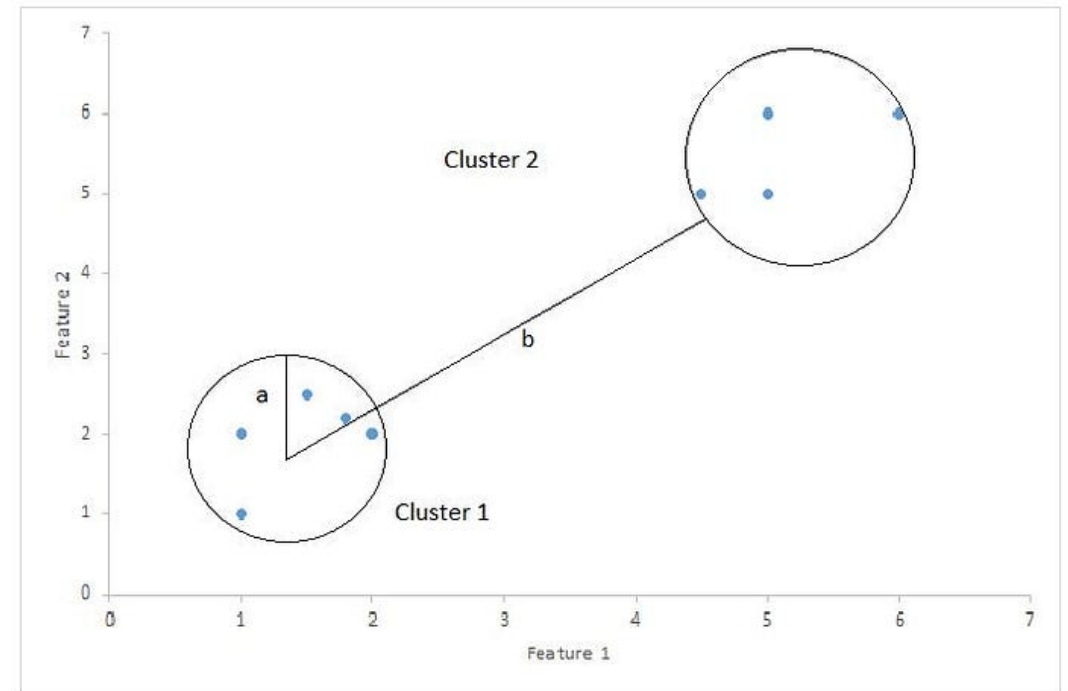
-1: significa que los grupos están asignados de forma incorrecta.

Índices internos

Coeficiente de Silhouette = $(b-a)/\max(a,b)$

a = distancia promedio dentro del grupo, i.e., la distancia promedio entre cada punto dentro de un grupo.

b = distancia promedio entre grupos, i.e., la distancia promedio entre todos los grupos.



Ejemplo en python

Conjunto de datos: *Clientes de un centro comercial*

Información del conjunto de datos:

Datos básicos sobre los clientes en un centro comercial. La puntuación de gasto se asigna al cliente en función de algunos parámetros, como el comportamiento del cliente y los datos de compra.

Información de atributos:

Para construir los datos, se tomaron cuatro características de los clientes: edad, sexo, ingresos anuales y puntuación de gastos

Ejemplo en python

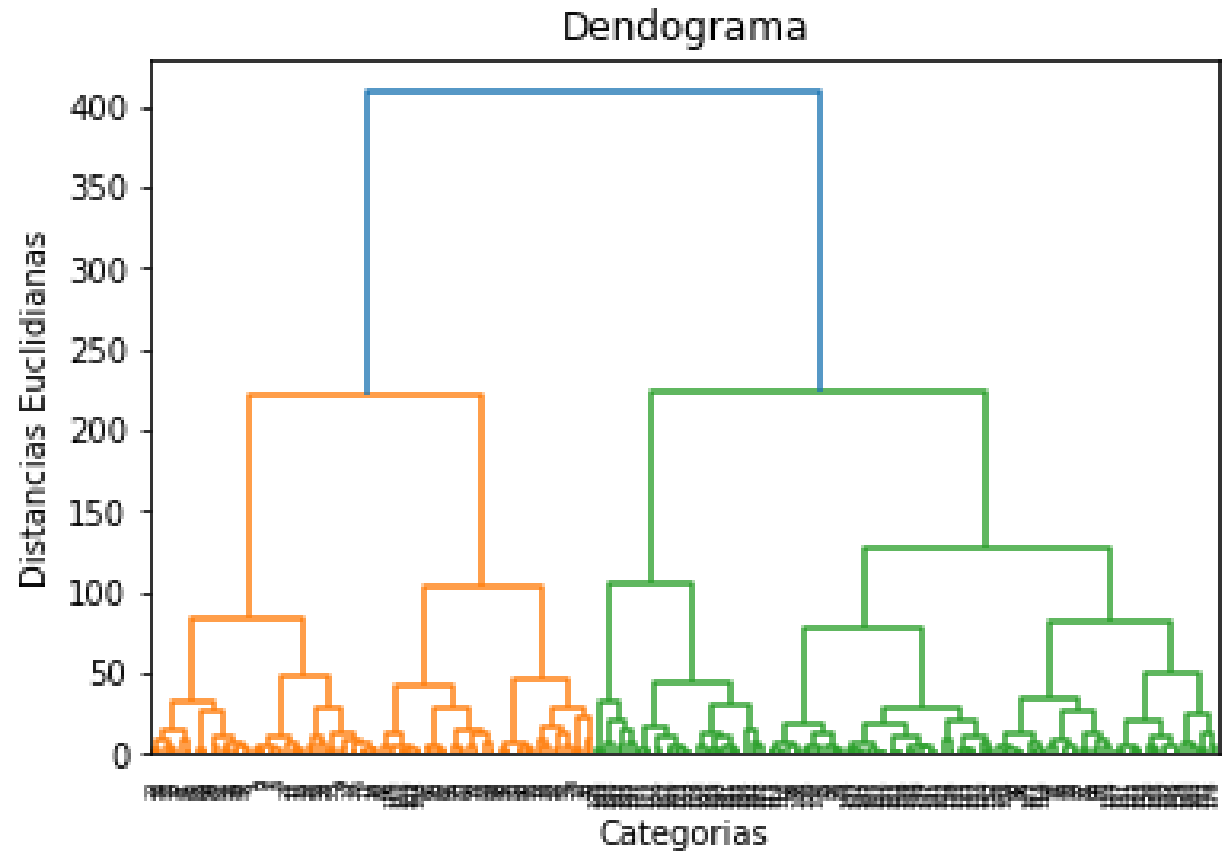
```
import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering
```

```
""" Clustering Jerárquico """
```

```
xCustomers = pd.read_csv('Mall_Customers.csv')
miniCustomers = xCustomers.iloc[:, [2, 3]].values
```

```
# Creamos el dendograma para encontrar el número
óptimo de clusters
```

```
dendrogram =
sch.dendrogram(sch.linkage(miniCustomers, method =
'ward'))
plt.title('Dendograma')
plt.xlabel('Categorías')
plt.ylabel('Distancias Euclidianas')
plt.show()
```



Ejemplo en python

Ajustando Clustering Jerárquico al conjunto de datos

```
hc = AgglomerativeClustering(n_clusters = 4,  
                             metric= 'euclidean',  
                             linkage = 'ward')
```

```
y_hc = hc.fit_predict(miniCustomers)
```

```
plt.scatter(miniCustomers[y_hc == 0, 0], miniCustomers[y_hc == 0, 1],  
            s = 100, c = 'red', label = 'Cluster 1')  
plt.scatter(miniCustomers[y_hc == 1, 0], miniCustomers[y_hc == 1, 1],  
            s = 100, c = 'blue', label = 'Cluster 2')  
plt.scatter(miniCustomers[y_hc == 2, 0], miniCustomers[y_hc == 2, 1],  
            s = 100, c = 'black', label = 'Cluster 3')  
plt.scatter(miniCustomers[y_hc == 3, 0], miniCustomers[y_hc == 3, 1],  
            s = 100, c = 'grey', label = 'Cluster 4')  
plt.title('Clusters of customers')  
plt.xlabel('Age')  
plt.ylabel('Annual Income (k$)')  
plt.legend()  
plt.show()
```



Ejemplo en python

```
from gap_statistic import OptimalK
```

```
""" K-means """
```

```
gs_obj = OptimalK(n_jobs=1, n_iter= 10)
```

```
n_clusters = gs_obj(xWheat, n_refs=50, cluster_array=np.arange(1, 15))
```

```
print('Optimal number of clusters: ', n_clusters)
```

Optimal number of clusters: 3

```
""" Clustering Jerárquico """
```

```
gs_obj = OptimalK(n_jobs=1, n_iter=20)
```

```
n_clusters = gs_obj(miniCustomers.astype('float'), n_refs=60,
```

```
cluster_array=np.arange(2, 10))
```

```
print('Optimal number of clusters: ', n_clusters)
```

Optimal number of clusters: 7

```
print(f'Silhouette Score(n=4): {silhouette_score(miniCustomers, y_hc)}')
```

Silhouette Score(n=4): 0.384

Density-based spatial clustering of applications with noise (DBSCAN)

DBSCAN significa Agrupamiento espacial basado en densidad de aplicaciones con ruido propuesto por Ester, Kriegel, Sander y Xu en 1996.

Es el método de aprendizaje no supervisado más utilizado entre los métodos de agrupamiento basados en densidad.

Tiene dos parámetros principales que deben elegirse cuidadosamente para obtener resultados óptimos:

- *eps*: el radio que define la vecindad alrededor de cada punto de datos.
- *min_samples*: el número mínimo de puntos de datos necesarios en una vecindad para que un punto de datos se considere un punto central.

DBSCAN clasifica los puntos de datos en tres categorías:

- *Punto central*: un punto que tiene al menos puntos de datos *min_samples* dentro de una vecindad de radio *eps*.
- *Punto fronterizo*: un punto que tiene menos de *min_samples*, pero al menos un punto central en su vecindad.
- *Ruido*: un punto que no es ni central ni fronterizo y que tiene menos de *min_samples* puntos en su vecindad.

Algoritmo DBSCAN

Entradas:

1. Un conjunto de datos $X = \{x_1, x_2, \dots, x_n\}$ en un espacio de alta dimensión $\mathbb{R}^d$
2. Parámetros: Radio del vecindario (eps), número mínimo de puntos en el vecindario para que un punto sea considerado central (min_samples) y la métrica de distancia a utilizar (por defecto, euclidiana)

Pasos del algoritmo:

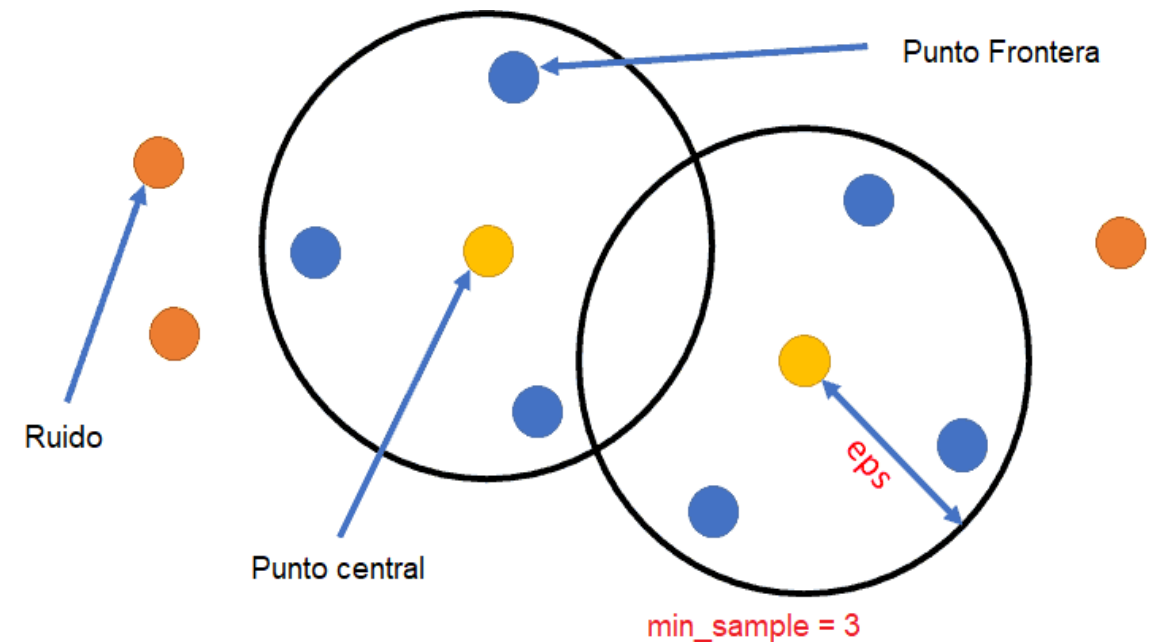
- a. Construcción del grafo de vecindad
- b. Cálculo de distancias geodésicas
- c. Aplicación de Escalamiento Multidimensional (MDS)

Salida:

1. Una representación $Y = \{y_1, y_2, \dots, y_n\}$ de los puntos originales con las etiquetas del cluster al que pertenece el punto correspondiente (-1 para ruido).
2. Puntos centrales: Lista de puntos identificados como centrales.
3. Puntos de borde: Lista de puntos identificados como de borde.
4. Puntos de ruido: Lista de puntos identificados como ruido.

Algoritmo DBSCAN

1. Se definen los parámetros *min_samples* y *eps*.
2. Se elige arbitrariamente un punto de partida a partir de los datos, su vecindad se define dentro de una distancia de *eps* y si hay al menos un número mínimo de puntos dentro de esta vecindad, entonces este punto de datos se clasifica como un punto central.
3. Se repite para todos los puntos. Encuentra los puntos de datos vecinos dentro de *eps* e identifica los puntos centrales. Para cada punto central, si ese punto no está asignado a un grupo, se crea un nuevo grupo.
4. Encuentra todos los puntos vecinos utilizando las métricas de distancia; y si hay al menos puntos *min_samples* dentro de *eps* para el punto de datos considerado, entonces todos estos puntos son parte del mismo grupo.
5. Se recorren los puntos no visitados. Si no forman parte de ningún grupo, se lo marca como valores atípicos/ruido.
6. Termina cuando se visitan todos los puntos de datos.



DBSCAN: Limitaciones, Complejidad y Variantes

Limitaciones:

- Sensibilidad a los parámetros: Los resultados pueden variar significativamente según la elección de ϵ y min_samples .
- Dificultad con clusters de densidades variables: DBSCAN puede tener problemas para identificar clusters con densidades muy diferentes.
- No determinístico: El orden de procesamiento de los puntos puede afectar los resultados.

Complejidad Computacional:

- Mejor caso: $O(n \log n)$ con una buena estructura de indexación espacial.
- Peor caso: $O(n^2)$ cuando la mayoría de los puntos están dentro de ϵ entre sí.

Variantes del algoritmo:

- OPTICS: Una generalización de DBSCAN que aborda la sensibilidad a los parámetros.
- HDBSCAN: Una versión jerárquica que puede manejar clusters de densidades variables.
- ST-DBSCAN: Una extensión para datos espacio-temporales.

DBSCAN: Interpretación de los resultados

1. *Número de clusters:* DBSCAN determina automáticamente el número de clusters, lo que puede ser útil cuando no se conoce este número a priori.
2. *Puntos de ruido:* Los puntos etiquetados como ruido (-1) pueden representar outliers o datos anómalos en tu conjunto.
3. *Forma de los clusters:* DBSCAN puede detectar clusters de formas arbitrarias, no solo circulares o elípticos.
4. *Densidad de los clusters:* Los clusters identificados tienen una densidad similar, determinada por `eps` y `min_simples`.
5. *Puntos centrales vs. de borde:* Los puntos centrales son más representativos del cluster, mientras que los puntos de borde pueden indicar áreas de transición entre clusters o hacia el ruido.
6. *Efecto de los parámetros:*
 - Un `eps` pequeño y `min_simples` grande resultará en clusters más densos y más puntos de ruido.
 - Un `eps` grande y `min_simples` pequeño puede llevar a menos clusters y menos puntos de ruido.
7. *Visualización:* En 2D o 3D, visualiza los clusters con diferentes colores y los puntos de ruido en negro para una fácil interpretación.

DBSCAN vs K-means

DBSCAN	K-means
No se necesita especificar el número de clústeres.	Es muy sensible al número de grupos, por lo que es necesario especificarlo.
Los clústeres formados pueden tener cualquier forma arbitraria.	Los clústeres formados son esféricos o de forma convexa
Puede funcionar bien con conjuntos de datos que tienen ruido y valores atípicos.	No funciona bien con datos atípicos. Valores atípicos pueden sesgar en gran medida los clústeres.
Se requieren dos parámetros para entrenar el modelo.	Solo se requiere un parámetro para entrenar el modelo.

DBSCAN



k-means



Ejemplo en python

```
from sklearn.cluster import DBSCAN
```

```
""" DBSCAN """
```

```
xCustomers = xCustomers[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
```

```
clustering = DBSCAN(eps=12.5, min_samples=4).fit(xCustomers)
```

```
DBSCAN_dataset = xCustomers.copy()
```

```
DBSCAN_dataset.loc[:, 'Cluster'] = clustering.labels_
```

```
DBSCAN_dataset.Cluster.value_counts().to_frame()
```

```
outliers = DBSCAN_dataset[DBSCAN_dataset['Cluster']== -1 ]
```

Ejemplo en python

```
fig2, (axes) = plt.subplots(1,2,figsize=(12,5))
```

```
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)',  
               data=DBSCAN_dataset[DBSCAN_dataset['Cluster']!=-1],  
               hue='Cluster', ax=axes[0], palette='Set2', legend='full', s=200)
```

```
sns.scatterplot(x='Age', y='Spending Score (1-100)',  
               data=DBSCAN_dataset[DBSCAN_dataset['Cluster']!= -1],  
               hue='Cluster', palette='Set2', ax=axes[1], legend='full', s=200)
```

```
axes[0].scatter(outliers['Annual Income (k$)'], outliers['Spending Score (1-100)'], s=10, label='outliers', c="k")
```

```
axes[1].scatter(outliers['Age'], outliers['Spending Score (1-100)'], s=10, label='outliers', c="k")
```

```
axes[0].legend()
```

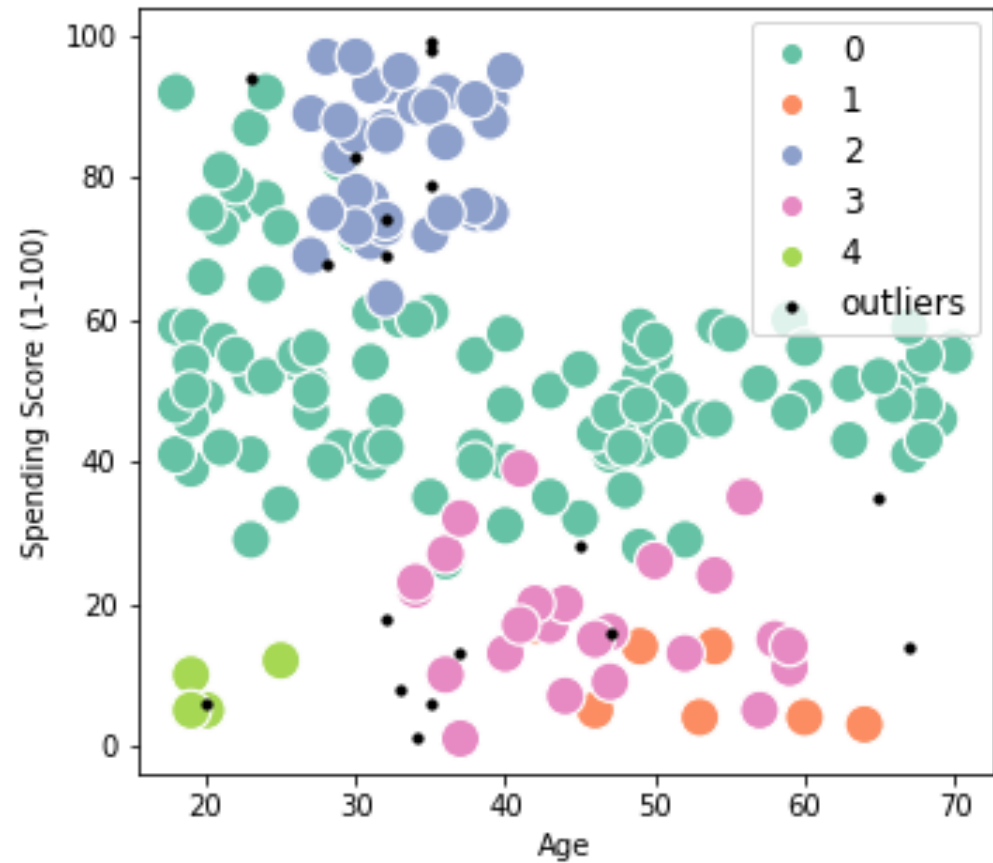
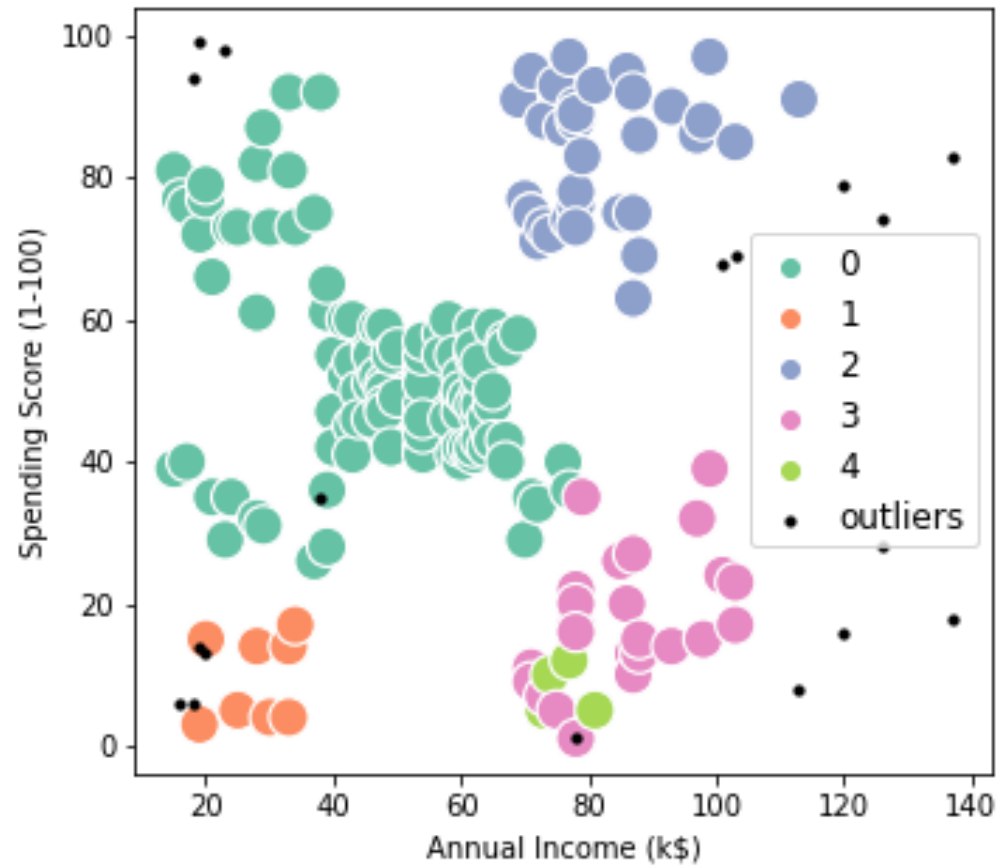
```
axes[1].legend()
```

```
plt.setp(axes[0].get_legend().get_texts(), fontsize='12')
```

```
plt.setp(axes[1].get_legend().get_texts(), fontsize='12')
```

```
plt.show()
```


Ejemplo en python



Hierarchical density-based spatial clustering of applications with noise (HDBSCAN)

El algoritmo HDBSCAN es una generalización de DBSCAN. Ambos algoritmos comienzan buscando la distancia núcleo para cada punto, esta es la distancia entre el punto y un número de vecinos cercanos definidos por el hiperparámetro `min_samples`.

Luego, se forma un dendograma con los posibles clusters. HDBSCAN intenta mantener a los clusters tan grandes como sea posible, por medio del control del hiperparámetro `min_cluster_size`.

La búsqueda del valor óptimo para los hiperparámetros que definen el desempeño del algoritmo se realiza maximizando el valor de la validez relativa para una dada configuración de tamaño mínimo de cluster y un número mínimo de muestras.

Algoritmo HDBSCAN

1. Transformar el espacio según la densidad.
2. Armar el mínimo árbol de expansión basado en el grafo de distancias ponderadas.
3. Construir clusters jerárquicos de componentes conectados.
4. Condensar la estructura jerárquica basándose en los clusters de menor tamaño.
5. Extraer los clusters más estables.